



Oppimistehtävä 3 - Transaktiot

Relaatiotietokannat

Jun Fengari, AG4042

Oppimistehtävä 3

Relaatiotietokannat, Jarkko Immonen

5.12.2025

Tietojenkäsittely

Sisältö

1	Ongelman Analyysi	2
2	Toteutus ja Havainnot.....	3
2.1	Dirty Read	3
2.2	Non-repeatable Read	3
2.3	Phantoms	4
3	Vertailu ja Analyysi	5
	Eristystasojen käsittelytapojen, käyttötarkoitusten ja tehokkuuksien analyysi.....	5
4	Reflektio ja tekoälyn käyttö	6
4.1	Reflektio	6
4.2	Tekoälyn Käyttö.....	7
	Lähteet	10

1 Ongelman Analyysi

1. Mihin tarvitaan transaktioita kirjasto-casessa?

Transaktiot ovat kokoelma tietokantaoperaatioita, jotka suoritetaan joko kokonaan tai ei ollenkaan. Näiden avulla säilytetään tietokannan eheys ja oikeellisuus myös silloin, kun useita muutoksia tehdään samanaikaisesti.

Kirjaston tietokannassa transaktioita tarvitaan, jotta tiedot pysyvät luotettavina, vaikka useat käyttäjät tekisi varauksia ja lainauksia samaan aikaan.

Käyttäjien on nähtävä oikea tieto siitä, onko kirjaa saatavilla, onko se lainattuna tai vapaana, ja heidän on voitava lainata ja palauttaa kirjoja ilman virhetilanteita. Ilman hyvin suunniteltuja transaktioita voisi syntyä tilanne, jossa kaksi käyttäjää lainaavat saman kirjan, josta on vain yksi fyysinen kopio saatavilla.

2. Miten ACID periaatteet näyttäytyvät tämän yhteydessä? Anna esimerkkejä.

Atomicity – Varmistaa, että transaktiot suoritetaan kokonaan tai ei lainkaan. Kirjastossa tämä tarkoittaa esimerkiksi sitä, että kirjan saatavuustiedossa ei ole mitään välitiloja, vain on oikea päivitetty tieto saatavuudesta (saatavilla, tai lainattu).

Consistency – Takaa, että transaktio suoriutuu sääntöjen mukaisesti ja sen jälkeen data ja tietokanta ovat eheässä tilassa ja oikeassa lopputilassa. Kirjasto tietokannan tilanteessa esimerkiksi ei saisi tulla virheitä jossa kirjan saatavuus menee negatiiviseen saldoon.

Isolation – Eristää samanaikaiset transaktiot toisistaan niin, etteivät näe toistensa keskeneräisiä muutoksia. Kirjastossa tämä estää tilanteen jossa käyttäjä näkee virheellistä tietoa saatavuudesta johtuen toisen käyttäjän keskeneräisestä lainauksesta.

Durability – Varmistaa, että transaktion onnistuessa muutos on pysyvä vaikka tulisi virhetilanne tai tietokanta kaatuisi. Esimerkiksi onnistuneet lainauksen jälkeen kirjan tila on pysyvä 'lainassa', riippumatta järjestelmän vikatilanteesta.

(Oluseye Jeremiah, 2025)

3. Miksi eristävyytasoja tarvitaan?

Eristävyytasoja tarvitaan varmistamaan, että useat käyttäjät pystyvät käyttämään kirjaston tietokantaa samanaikaisesti ilman virheitä tai datan eheysongelmia.

Eristävyytasot ovat tärkeitä kaikissa järjestelmissä, joissa tapahtuu rinnakkaisia transaktioita, esimerkiksi pankeissa. Transaktioiden avulla estetään tyypillisiä ongelmia ja varmistetaan että tietokanta pysyy ennustettavana ja turvallisena usealle käyttäjälle yhtä aikaan.

(”Isolation (Database Systems)”, 2025)

4. Millaisia ongelmia voi syntyä jos käyttäjät päivittävät samoja tietoja yhtä aikaa?

Samanaikaisissa päivityksissä voi syntyä useita rinnakkaisuuden (concurrency) ongelmia:

Dirty Read – tai Temporary Update Problem, on tapaus jossa transaktio lukee ja ottaa käyttöön toisen transaktion tekemän keskeneräisen muutoksen. Tämä muodustuu ongelmaksi kun toisen transaktion muutokset perutaan – luettu tieto on virheellinen.

Non-Repeatable Read – on tilanne jossa transaktion aikana luetaan rivin kahdesti, mutta arvo muuttuu toisella luku-kerralla, johtuen toisen transaktion väliajassa tapahtuneesta päivitys-toimeenpiteistä.

Phantom – on tilanne jossa transaktio suorittaa saman haun kahdesti, mutta toisella kerralla tulos sisältää uusia tai poistuneita rivejä, johtuen toisen transaktion välissä lisäämästä tai poistamasta datasta.

On myös tilanteita jolloin transaktion päivityksen menevät päällekkäin siten, että toisen tekemät muutokset saattaa yliajautua kokonaan. Lopputulos on, että toisen transaktion päivitys katoaa, eli **Lost Update Problem**. (Concurrency Problems in DBMS Transactions, 2025).

2 Toteutus ja Havainnot

2.1 Dirty Read

- Käyttäjä yksi muokkaa lainaus-taulua UPDATE-operaatiolla:
`SET autocommit=0;`
`START TRANSACTION;`
`UPDATE lainaus SET erapaiva = "2019-08-30" WHERE lainaus=1;`
- Käyttäjä kaksi aloittaa oman transaktion READ UNCOMMITTED-eristyvyydellä, ja odotan, että hän pystyy näkemään käyttäjän 1 tallentamattomat muutokset.
`SET autocommit=0;`
`SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;`
`START TRANSACTION;`
`SELECT * FROM lainaus WHERE lainaus = 1;`
- Seuraavaksi käyttäjä kaksi kokeilee READ COMMITTED, REPEATABLE READ ja SERIALIZABLE, ja odotan, että hän ei pysty näkemään muutoksia.

Tulokset:

Tapaus	Eristystaso	Miten kävi?	Havainnot & Johtopäätökset
Dirty Read	READ UNCOMMITTED	Keskeneräistä muutosta ei näkynyt	Ei ole hyväksyttävä eristyvyytaso kirjastolle, mutta InnoDB estää dirty read.
Dirty Read	READ COMMITTED	Keskeneräistä muutosta ei näkynyt	OK dirty-readin estoon.
Dirty Read	REPEATABLE READ	Keskeneräistä muutosta ei näkynyt	OK dirty-readin estoon
Dirty Read	SERIALIZABLE	Keskeneräistä muutosta ei näkynyt	OK dirty-readin estoon

2.2 Non-repeatable Read

- Käyttäjä yksi aloittaa transaktion eri eristyvyytasoilla ja hakee ensin nimeke-taulusta kerran tietyn kirjan tietoja.
`SET autocommit=0;`
`SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;`
`START TRANSACTION;`
`SELECT * FROM nimeke WHERE nimeke = '0000-1111-222';`
`-- tauko`
`SELECT * FROM nimeke WHERE nimeke = '0000-1111-222';`
- Tauon aikana käyttäjä kaksi tekee muutoksen samaan kirjaan UPDATE-operaattorilla:
`START TRANSACTION;`
`UPDATE nimeke SET nimi = 'Lintubongaaja' WHERE nimeke = '0000-1111-222';`
`COMMIT;`
 Jonka jälkeen käyttäjä yksi hakee saman tiedon uudestaan.

- Odotan että READ UNCOMMITTED ja READ COMMITTED näyttävät muutoksen, ja epäilen, että eristyyvyystasot REPEATABLE READ ja SERIALIZABLE mahdollisesti estävät tämän.

Tulokset:

Tapaus	Eristystaso	Miten kävi?	Havainnot & Johtopäätökset
Non-repeatable Read	READ UNCOMMITTED	Commit muutos näkyy	Ei hyväksyttävä.
Non-repeatable Read	READ COMMITTED	Commit muutos näkyy	Ei hyväksyttävä, estää dirty read mutta non-repeatable read on mahdollinen.
Non-repeatable Read	REPEATABLE READ	Commit muutos ei näkynyt	OK, estää non-repeatable read.
Non-repeatable Read	SERIALIZABLE	Commit muutos ei näkynyt	OK, estää non-repeatable read.

2.3 Phantoms

- Käyttäjä yksi aloittaa transaktion eri eristyyvyystasoilla ja hakee kaikki kirjat nimeke-aulusta luokituksesta 55.5:

```
SET autocommit=0;
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
SELECT * FROM nimeke WHERE luokitus = 55.5;
-- tauko
SELECT * FROM nimeke WHERE luokitus = 55.5;
```
- Käyttäjä kaksi lisää kirjan INSERT operaattorilla:

```
START TRANSACTION;
INSERT INTO nimeke VALUES ('4444-2345-888', 'Suuri Lintukirja', NULL, 55.5, 4, 'Uusi Kirja 1', 'linnut, opas');
```

COMMIT;

Jonka jälkeen käyttäjä yksi hakee saman luokituksen kirjat.
- Odotan, että READ UNCOMMITTED ja READ COMMITTED -eristyyvyystasot eivät estä phantom readia, REPEATABLE READ mahdollisesti estää, ja SERIALIZED pitäisi estää.

Tulokset:

READ COMMITTED -eristyyvyystasolla kohdalla ilmeni tilanne, jossa tulokset eivät vastanneet odotuksia: eristyyvyystaso ei käyttäytynyt täysin oman logikan tai yleisten sääntöjen mukaisesti. Teoriassa READ COMMITTED pitäisi näyttää phantom read -tilanteet, mutta näin ei käynyt.

Tutkin ja löysin MySQL dokumentaatiosta (*MySQL :: MySQL 8.4 Reference Manual :: 17.7.1 InnoDB Locking*, ei pvm.) lisätietoja Gap Locks -osiosta. Dokumentaatio selittää että gap lock saattaa estää uusien rivien lisäystä transaktioiden aikana erityisesti silloin, kun InnoDB käyttää indeksihakua ja next-key lukitusta. Tämä voisi selittää, miksi phantom read ei näkynyt ensimmäiselle käyttäjälle READ COMMITTED -eristyyvyystasolla.

Tapaus	Eristystaso	Miten kävi?	Havainnot & Johtopäätökset
Phantoms	READ UNCOMMITTED	Commit muutos näkyy	Ei hyväksyttävä.
Phantoms	READ COMMITTED	Commit muutos ei näkynyt	Mahdollinen Gap-Lock tilanne; normaalisti muutos pitäisi näkyä.
Phantoms	REPEATABLE READ	Commit muutos ei näkynyt	OK, estää phantom readin.
Phantoms	SERIALIZABLE	Commit muutos ei näkynyt	Ok, estää kaikki testatut concurrency-virheet.

3 Vertailu ja Analyysi

Eristystasojen käsittelytapojen, käyttötarkoitusten ja tehokkuuksien analyysi

Eristyvyytason valinta riippuu sovelluksen käyttötarkoituksesta, käyttäjämääristä, data eheyden vaatimuksista sekä tehokkuuden ja suorituskyvyn tarpeesta. Tässä kirjoitelmassa tarkastelen kunkin eristyvyytason konseptuaalista ja teknistä toimintaa, niiden vaikutusta tehokkuuteen ja tietojen johdonmukaisuuteen sekä tyypillisiä käyttötapauksia.

Eristyvyytason tehokkuus heikkenee sitä mukaan kun eristyvyyden taso kasvaa. READ UNCOMMITTED on tehokkain eristyvyytaso, mutta sallii kaikki yleiset samanaikaisuuden virheet, kun taas SERIALIZABLE on kaikista raskain suorituskyvyn kannalta, mutta estää kaikki tunnetut samanaikaisuuden virheet.

Valitsin analyysin aiheeksi Non-Repeatable Read -ongelmaan, johon tutustuin ylemmässä tehtävässä, 2.2. Tarkastelen sen toimintaa syvemmin, sekä siihen liittyvää Repeatable Read eristyvyytasoja, muiden lisäksi.

READ UNCOMMITTED -eristyvyytasolla SELECT-lauseet suoritetaan ilman lukituksia (non-locking fashion). Se toimii monilta osin samankaltaisesti kuin READ COMMITTED, mutta ei estä dirty readejä, minkä vuoksi palautetut tulokset voivat olla epäjohdonmukaisia. READ UNCOMMITTED sallii myös kaikki muut samanaikaisuuden virheet, joita havaitsin tehtävissä.

Tämän vuoksi tätä eristyvyytasoja ei yleensä suositella käytettäväksi sovelluksissa jossa data eheys on tärkeää. Se soveltuu sovelluksiin joissa suorituskky on tärkeämpää kuin tietojen tarkkuus, kuten raportoinnissa, analytiikassa ja lokitietojen käsittelyssä.

READ UNCOMMITTED on kaikista eristyvyytasoista nopein, koska se lukee suoraan viimeisimman tietueen tilan, riippumatta COMMIT-statuksesta. Tällä tasolla ei ole lukitsemista eikä versiointia, toisin kun muissa eristyvyytasoissa, jotka käyttävät erilaisia lukitusstrategioita ja versionhallintaa datan eheyden varmistamiseksi. Tämä tarkoittaa että READ UNCOMMITTED kuormittaa järjestelmää vähiten, mutta tinkii datan eheydestä.

READ COMMITTED -eristyvyytasolla jokainen SELECT-lause lukee aina viimeisimmän commitatun version tiedoista, eli se muodostaa uuden snapshotin jokaisen kyselyn ajaksi. Tämä eristyvyytaso käyttää tilanteesta ja kyselystä riippuen lukitusta.

InnoDB:ssä esimerkiksi UPDATE-komento lukitsee rivin jota yritetään muokata niin kauan, kunnes transaktio lopetetaan (COMMIT tai ROLLBACK). Lukituksen aikana muut transaktiot eivät pääse muokkaamaan samaa riviä. Read Committed pystyy vapauttamaan lukot riveistä, jota transaktio ei muuta, joka mahdollistaa ns. semi-consistent read tilanteen, jota voidaan käyttää WHERE-ehtojen arvionnissa.

READ COMMITTED -tasolla on myös huomioitava gap-lock-ilmiö, mikä tuli mahdollisesti tehtävässä 2.3 vastaan. Gap lockeja ei käytetä SELECT-lauseissa, mutta voivat ilmestyä kun suoritetaan muutoksia sarakkeisiin jossa järjestelmä joutuu tarkistamaan indeksejä, joka aiheuttaa ajoittain kontekstisidonnaista käyttäytymistä.

READ COMMITTED sopii parhaiten sovelluksiin, jotka tarvitsevat tasapainon datan johdonmukaisuuden ja suorituskyvyn välillä. Sitä käytetään monissa OLTP-järjestelmissä (online transaction processing), kuten esimerkiksi varauspalveluissa, verkkokaupoissa, kassajärjestelmissä, eli sovelluksissa jossa käyttäjät lukevat, lisäävät ja muokkaavat pieniä datamääriä samanaikaisesti.

REPEATABLE READ -eristyvyytaso estää tehokkaasti non-repeatable read -ongelman, jonka valitsin tähän analyysiin.

Non-repeatable read tapahtuu kun transaktion sisällä luetaan sama rivi kaksi kertaa, mutta toisella kerralla

riviin on tullut muutoksia toisesta transaktiosta johtuen. Tämä ilmiö oli myös mahdollinen READ UNCOMMITTED ja READ COMMITTED eristystasojilla, kuten huomasin tehtävissäni. REPEATABLE READ -eristystaso luo snapshotin transaktion alkaessa, johon kaikki SELECT-lauseet perustuvat transaktion aikana, riippumatta toisten transaktioiden tekemistä muutoksista. Tämä tarkoittaa että taustalla voi olla useampaa eri versiota samasta rivistä ja transaktio aina käyttää sitä tiettyä versiota kunnes transaktio lopetetaan.

REPEATABLE READ käyttää erilaista lukitsemiskäyttäytymistä tilanteen ja kyselyn rakenteen mukaan. Jos WHERE-ehdossa käytetty sarakke on indeksoitu, InnoDB lukitsee vain tarvittavat rivit. Jos indeksiä ei ole olemassa, taulu saatetaan skannata kokonaisuudessaan mikä johtaa laajempiin lukituksiin. Tämä selittää miksi REPEATABLE READ saattaa olla tehokas joissain rakenteissa ja raskaampi toisissa. REPEATABLE READ -eristystaso sopii hyvin sovelluksiin, joissa tietojen pysyvä johdonmukaisuus on tärkeä, kuten pankkisovelluksiin, raportointiin ja auditointiin. Se on myös sopiva meidän kirjastotietokantaan, sillä se estää suurimmat samanaikaisuuden ongelmat, ilman että on liian raskas.

SERIALIZABLE-eristystaso estää kaikki yleiset samanaikaisuuden virheet, mutta tekee sen tehokkuuden kustannuksella. Se on kaikista raskain eristystaso, sillä sen käsittelytapa johtaa laajoihin lukituksiin jotka estää samanaikaisuuden ongelmien syntymistä. Tässä on kuitenkin riskinsä, sillä transaktioiden 'deadlockit' ovat mahdollisia jossa tietokannan transaktiot jäävät lukkoon koska ne odottavat toisien transaktioiden vapauttavan tarvittavia rivejä.

Tästä syystä, SERIALIZABLE-eristystasoa käytetään vaan sovelluksissa ja järjestelmissä jossa vaaditaan täydellistä tietojen eheyttä. Tällaisia sovelluksia ovat esimerkiksi pankkijärjestelmät, monimutkaiset eräajot (batch processing jobs), tietyt varastonhallintajärjestelmät sekä muut kriittiset transaktioprosessit jossa ei saa tapahtua virheitä.

Tähän vastaukseen käytin seuraavia lähteitä oppimismateriaalina:

- *MySQL :: MySQL 8.4 Reference Manual :: 17.7.2.1 Transaction Isolation Levels*, ei pvm.
- Brian Morrison II, 2024
- *What are SQL Isolation Levels and How to Choose?*, 2024

4 Reflektio ja tekoälyn käyttö

4.1 Reflektio

Opin paljon uutta transaktioista ja eristystasojista, ja tämä ymmärrys auttoi hahmottamaan miten niitä voidaan analysoida ja soveltaa järjestelmiin ja sovelluksiin. Lisäksi se auttoi näkemään kokonaisuuden muiden oppimistehtävien kanssa.

Vaikka eristystasot vaikuttivat alussa yksinkertaisilta, järjestelmätasolla tapahtuu monimutkaisia prosesseja, ja InnoDB:n päätökset eivät aina olleet yhtä selkeitä kuin alussa oletin. Tämä ilmeni tehtävissä, joissa eristystaso ei käyttäytynyt täysin odotuksien mukaisesti.

Haastavinta tässä oppimistehtävässä oli ymmärtää, miten simuloida oikein kaksi käyttäjää. Vaikka onnistuin simulaatiossa, operaatiot piti suprittaa hyvin systemaattisesti, muuten tulokset saattoivat vääristyä. Oli välillä haastavaa erottaa, johtuiko väärä tulos omista virheistä vai oliko kyseessä tunnettu poikkeustilanne. Myös eristystasojen konseptuaalinen ymmärtäminen, varsinkin eri lukitus ja versiointimekanismien hahmoittaminen oli haastavaa. Käytännön kokeilu antoi hyvän kuvan yleisestä käyttäytymisestä. Käytin tämän oppimistehtävän tekemiseen 1.5 viikkoa, yhteensä noin 11 tuntia.

4.2 Tekoälyn Käyttö

Käytin tekoälyä tässä tehtävässä ymmärtämään tehtävänantoa ja saamaan laajempaa käsitystä siitä, mitä tehtävänannossa haettiin.

Eniten tekoälyä käytin toteutus-osiossa, jossa kohtasin ongelmia kahden käyttäjän simuloinnissa ja tarvitsin apua virheellisten tulosten jäljittämiseksi. Tekoäly toimi tällöin ongelmanratkaisun tukena, eikä vaikuttanut varsinaisiin vastauksiini.

Pyysin tekoälyltä myös lukumateriaalia teoriaosioon, joiden pohjalta pystyin tekemään analyysin.

Kirjoittamisen jälkeen tarkistin tekoälyn avulla, oliko analyysini teorian mukainen ja oikein perustettu, ja tarvittaessa täydensin tietojani lähteistä varmistaakseni tekoälyn ehdotuksen totuudenmukaisuutta.

Lopuksi käytin tekoälyä koko tehtävän kieliasun tarkistamiseen.

Sisällytin relevantteja kuvankaappauksia alla.

Ennen osaa 2: tehtävän ja transaktioiden ymmärtämistä, sekä syntaksia:

✓ 2. How to set the isolation level (eristyvyystaso)

MySQL syntax

sql

Copy code

```
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

Or whichever level you want:

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE

Examples:

sql

Copy code

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```



Great question — and this is one that confuses a lot of people at first.

✓ Why you *don't have to* set the isolation level for Transaction A

In almost all tests (dirty read, non-repeatable read, phantom read), **Transaction A** is only changing data — it's *not reading anything important*.

Isolation levels mostly affect **how a transaction reads data**, not how it writes.

So:

- Transaction A is just doing:
`START TRANSACTION; UPDATE ...;` and then staying uncommitted.
- It doesn't matter what its isolation level is, because it has no read-related rules to enforce.
- What matters is **how Transaction B sees (or doesn't see) A's uncommitted changes**.

Transaction B is the one doing the reading, so the isolation level is relevant only for B.

Ongelmanratkaisua, kun en osannut simuloida kaksi käyttäjää kunnolla:

2 How to properly simulate two users

You need two completely separate connections:

1. **Connection 1** (User 1 / Transaction A):
 - Set isolation level, start transaction, read the row.
2. **Connection 2** (User 2 / Transaction B):
 - Start transaction, update the row, commit.
3. **Back in Connection 1:**
 - Run the second read.
 - Now you will see **old value** at REPEATABLE READ (or SERIALIZABLE).

Tip: In Workbench, use **Database** → **Connect to Database** → **New Connection** or click the “+” button to open a completely new connection. Tabs alone are not enough.

1. Always close or reset transactions before starting new tests:

```
sql
ROLLBACK; -- if you had uncommitted changes
```

2. Clear the SQL editor or open a new connection if things behave weirdly.
3. Run statements in clean order — don't leave `START TRANSACTION` or old comments above your command unless you really want a transaction to start.
4. Verify the result explicitly with a fresh `SELECT` after each delete or insert.

Ongelmia kohdassa 2.3 jossa eristyyvyystasot ja phantom read ei toiminut odotetusti. Kuvassa on tekoälyn ehdotus, perehdyin syvemmin dokumentaatioon tämän ehdotuksen pohjalla, ja sitten kyselin lisää tästä lukitsemiskäyttäytymisestä.

Why your INSERT works but the phantom does not show

Your INSERT is valid and correct:

```
sql
INSERT INTO nimeke VALUES ('4444-2345-888', 'Suuri Lintukirja', NULL, 55.5, 4, 'Uusi Kirja 1', 'li
```

The problem is not the INSERT.

The problem is the *test conditions* — MySQL uses locking behavior that prevents the phantom under certain table/index setups.

InnoDB will NOT show new rows inserted by another transaction unless:

- Those rows were committed (they were) **AND**
- They match the index condition exactly **AND**
- Your first SELECT did not already use a range lock that prevents the phantom

And that last part is the important one:

Your WHERE clause involves a non-indexed column

MySQL/InnoDB sometimes uses **Next-Key Locks** even in READ COMMITTED depending on the table structure.

If `luokitus` is not indexed, InnoDB may lock larger ranges of rows (gap locks) even in READ COMMITTED, preventing new rows from appearing in the range.

So in your case:

- `SELECT * FROM nimeke WHERE luokitus = 55.5;`
- If no index exists on `luokitus`, InnoDB scans with range locks.
- This blocks the phantom from being visible to User 1.

This is **expected** for MySQL's implementation — it is not pure ANSI READ COMMITTED.

Pyysin osioon 3 lisää lukumateriaalia jotta ymmärtäisin eristävyytasoja ja prosessointi paremmin ennen kuin vastasin esitettyihin kysymyksiin.

Yes — absolutely. There *are* good, reliable sources you can read to understand how InnoDB / MySQL handles query execution, isolation levels, locking and snapshot-based reads. Reading them will help you answer conceptually what's asked in your assignment. Below are some of the best ones to start with (with explanation what you'll learn from each).



Recommended Official Documentation & Articles

What you'll learn / Why it helps	Source / Link
Overview of the four isolation levels (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE) as supported by InnoDB; how default isolation is REPEATABLE READ. Good base for comparing levels. dev.mysql.com +1	Transaction Isolation Levels — MySQL Reference Manual dev.mysql.com
How InnoDB implements "consistent nonlocking reads" (snapshot-based reads / MVCC). Explains why SELECTs may or may not see other transactions' changes depending on isolation level. dev.mysql.com +1	Consistent Nonlocking Reads — MySQL Reference Manual dev.mysql.com
Explanation of locking behavior: when InnoDB uses index locks vs when it uses "gap locks" or "next-key locks," and how this affects phantom rows / concurrent inserts. dev.mysql.com +1	InnoDB Locking (including gap lock / next-key lock discussion) — MySQL docs dev.mysql.com +1
A readable summary of the tradeoffs between isolation level ↓ consistency vs accuracy, performance vs safety, and in which contexts different levels make sense	"MySQL and InnoDB Storage Engine Summary" — Software article dev.mysql.com

Kysyin myös selitystä 3 osan ensimmäiseen kysymykseen kun kysyttiin konseptuaalista selitystä, että mitä siihen kuuluisi sisältyä:

2 Conceptual processing per eristävyytaso

- This is how MySQL actually *handles your queries internally* depending on the isolation level.
- Things to describe include:
 - Does the SELECT see a snapshot or the most recent committed data?
 - Are locks acquired (row locks, gap locks, next-key locks)?
 - How does the isolation level prevent (or allow) dirty reads, non-repeatable reads, and phantoms?
- Essentially, this is *why* you observed the results you did in part 2.

Palasin osaan 2 & 3 kysymään uudestaan lukituksien toiminnasta:

★ Key fact you need for your report

Even if **luokitus** had *no index*, the table **still has a PRIMARY KEY** (the *nimeke* column in your inserts).

And MySQL **always** uses the primary key index to scan the table when no suitable secondary index exists.

This means:

- Your query `WHERE luokitus = 55.5` caused a **full index scan on the PRIMARY KEY**.
- During that scan, InnoDB applied **next-key locks** on the primary key index ranges.
- Next-key locks include **gap locks**, so they blocked inserts into the scanned key range.
- As a result → **no phantom appeared**, even at READ COMMITTED.

Lähteet

Brian Morrison II. (2024, tammikuuta 8). *MySQL isolation levels and how they work—PlanetScale*.

<https://planetscale.com/blog/mysql-isolation-levels-and-how-they-work>

Concurrency problems in DBMS Transactions. (2025, kesäkuuta 12). GeeksforGeeks.

<https://www.geeksforgeeks.org/dbms/concurrency-problems-in-dbms-transactions/>

Isolation (database systems). (2025). Teoksessa *Wikipedia*. [https://en.wikipedia.org/w/index.php?title=Iso-](https://en.wikipedia.org/w/index.php?title=Isolation_(database_systems)&oldid=1288643596)

[lation_\(database_systems\)&oldid=1288643596](https://en.wikipedia.org/w/index.php?title=Isolation_(database_systems)&oldid=1288643596)

MySQL :: MySQL 8.4 Reference Manual: 17.7.1 InnoDB Locking. (ei pvm.). Noudettu 1. joulukuuta 2025,

osoitteesta <https://dev.mysql.com/doc/refman/8.4/en/innodb-locking.html>

MySQL :: MySQL 8.4 Reference Manual: 17.7.2.1 Transaction Isolation Levels. (ei pvm.). Noudettu 1.

joulukuuta 2025, osoitteesta <https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-isolation-levels.html>

Oluseye Jeremiah. (2025, tammikuuta 14). *Understanding SQL Transactions: A Comprehensive Guide*. Data-

Camp. <https://www.datacamp.com/tutorial/sql-transactions>

What are SQL Isolation Levels and How to Choose? (2024, kesäkuuta 7).

<https://www.pingcap.com/article/what-are-sql-isolation-levels-and-how-to-choose/>

